Session 8 Project Guide

EAG V3 Assignment 8 - Multi-Agent Growing-Graph Orchestrator

This guide explains what the project does, how the code is organized, and how the five assignment parts fit together. For run commands see ASSIGNMENT_STATUS.md. For every saved run see SESSIONS_CATALOG.pdf.

---

1. What this project is

Session 8 replaces a single-loop agent with a directed acyclic graph (DAG) of skills. Each node is an LLM-backed role (Planner, Researcher, Coder, etc.). Edges carry results between nodes. The graph grows at runtime when skills finish and the orchestrator splices in successors.

Problem of choice: Compare and rank information from multiple independent sources - parallel web research, optional Python computation in a sandbox, a comparator skill to pick winners, and a critic to validate upstream output before the final answer.

Design rule: The Planner names skills, never tools. Tools are declared per skill in agent_config.yaml (tools_allowed).

---

2. Two processes you run

Process	Directory	Port	Role
LLM Gateway	`S8SharedCode/gateway/`	8108	Chat, routing, embeddings
Agent	`S8SharedCode/code/`	-	DAG orchestrator, skills, sessions

Terminal 1:

```
cd S8SharedCode/gateway && uv run main.py
```

Terminal 2:

```
cd S8SharedCode/code && uv run python flow.py "your question"
```

---

3. How one query runs (end to end)

1. **CLI** - `flow.py` calls `Executor.run()`.
2. **Session** - ID like `s8-abc12345`; query saved to `state/sessions/<sid>/query.txt`.
3. **Memory** - FAISS hits read once; injected into every skill prompt.
4. **Seed** - Single `planner` node with input `USER_QUERY`.
5. **Loop** until no pending work:

* Find **ready nodes** (all predecessors `complete` or `skipped`).

* Run ready nodes **in parallel** via `asyncio.gather`.
* On success: store result, call `graph.extend_from()` to add successors.
* On failure: `recovery.py` classifies and may splice a new Planner node.

6. **Answer** - Formatter's `output.final_answer`, or fallback from last node.

Simple shapes:

```
hello:     planner -> formatter
research:  planner -> researcher -> formatter
code:      planner -> coder -> sandbox_executor -> formatter
parallel:  planner -> [researcher, researcher, researcher] -> merge -> formatter
```

---

4. Key files map

```
Assignment 8/
  ASSIGNMENT_STATUS.md     how to run + rubric status
  docs/PROJECT_GUIDE.md    this document
  generate_pdf.py          markdown -> PDF
  generate_sessions_report.py   sessions -> catalog markdown

S8SharedCode/code/
  flow.py                  orchestrator (Graph + Executor) - do not edit for assignment
  skills.py                skill registry, prompt render, run_skill()
  recovery.py              failure classification + critic-fail splice
  persistence.py           graph.json + per-node JSON on disk
  gateway.py               HTTP client to gateway :8108
  mcp_runner.py            multi-turn tool-use loop
  sandbox.py               subprocess Python runner
  agent_config.yaml        skills catalogue (yaml + prompts)
  prompts/*.md             one prompt file per skill
  state/sessions/<sid>/    persisted runs (graph.json, nodes/, query.txt)
  scripts/run_assignment.sh   run parts 1-5
  replay.py                walk a session node-by-node
  visualize_graph.py       HTML DAG viewer
```

---

5. Skills catalogue

Skill	Tools	Purpose
planner	none	Emits initial DAG; recovery replans on failure
retriever	search_knowledge	FAISS / indexed corpus
researcher	web_search, fetch_url	Web research, URL fetch
distiller	none	Structured fields from text; **auto-critic** on ou
summariser	none	Condense long content
comparator	none	Compare upstream outputs, rank / pick winner (**ne
critic	none	Pass/fail verdict on upstream output
coder	none	Emits Python JSON; routes to sandbox via `internal

sandbox_executor	none	Runs coder's Python in subprocess
formatter	none	Terminal node; `final_answer` returned to user

Graph growth mechanisms:

* **Dynamic successors** - skill returns `AgentResult.successors` (Planner's plan).
* **internal_successors** - coder always gets `sandbox_executor` after success.
* **Critic auto-insert** - distiller edges get a critic node before children.
* **Recovery** - failed nodes or critic-fail may splice a new `planner` node.

---

6. Assignment parts 1-5 (evidence sessions)

Part	What	Key session
1	Base queries hello, A, I, J, K	hello `s8-7f8a75bf`, A `s8-5c7b354b`, I `s8-30cc4b
2	Parallel fan-out (3+ branches); wall-clock = max(	`s8-453bce58` (CRISPR/mRNA/solar)
3	Critic pass + fail + planner recovery	pass `s8-4fd467a0`, fail `s8-418393c0`
4	Coder prompt + sandbox on compute query	`s8-30cc4b2c` (query I)
5	New skill `comparator` in yaml + prompt	`s8-a6972d7c`

Regression: uv run pytest tests/test_recovery.py - 22 passed.

---

7. How to run the assignment

```
cd S8SharedCode/code

./scripts/run_assignment.sh 1 hello
./scripts/run_assignment.sh 1 A
./scripts/run_assignment.sh 1 I
./scripts/run_assignment.sh 1 J
./scripts/run_assignment.sh 1 K
./scripts/run_assignment.sh 2
./scripts/run_assignment.sh 3 pass
./scripts/run_assignment.sh 3 fail
./scripts/run_assignment.sh 4
./scripts/run_assignment.sh 5
```

Query K: kill mid-run, then uv run python flow.py --resume <sid>.

---

8. Inspect and visualize

```
uv run python replay.py <session_id>
uv run python visualize_graph.py <session_id> --open
uv run python scripts/verify_parallel_timing.py <session_id>
```

Session data lives in state/sessions/<sid>/:

* `query.txt` - user question
* `graph.json` - full DAG (NetworkX node_link format)
* `nodes/n_*.json` - per-node prompts and results

---

9. Architecture constraints (course)

* Skills = `agent_config.yaml` entry + `prompts/<skill>.md`.
* Planner emits the graph; Executor runs it; Critic sits between flagged producer and successor.
* Adding a skill is a yaml edit + prompt file - **not** an Executor change.
* Supporting fixes in `skills.py` / `mcp_runner.py` are reportable; `flow.py` should stay intact.

---

10. Further reading

Document	Content
`README.md`	Full architecture (long)
`ASSIGNMENT_STATUS.md`	Run commands + checklist
`CODE_EXPLANATION.pdf`	README as PDF
`SESSIONS_CATALOG.pdf`	All saved sessions
`PARALLEL_FANOUT.md`	Part 2 query design
`CRITIC_VERDICT.md`	Part 3 query design